

Teil 4: Automation

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Agenda

1. Build Management

2. DevOps

3. Monitoring

Knight Capital

- SMARS high-speed trading system ported to new environment
- Feature flag for debug-only buy-high-sell-low algorithm reused for new code
- Massive refactoring of the codebase
- Insufficient tests + manual deployment + little telemetry + no rollback plan
- $\Rightarrow$ Buy high, sell low!
- Bought shares re-sold at **\$440M loss**

$\Rightarrow$ **How could this have been prevented?**



Build Management

- **Self-contained:** All metadata describing the build process can be distributed alongside the source code.
- **Universal:** The build process can be executed on any supported platform. There should be no single “build guru” that knows the mysteries of the build system.
- **Automated:** No intervention during the process is necessary (no GUI interaction, no questions, ...). Preparation (installation of tools, dependencies, etc.) might still be required.
- **Simple:** The process can be triggered using as little steps as possible – ideally 1!
- **Well-documented:** If the build can or must be customized (feature flags, compiler settings, etc.), some form of documentation should be present.

Build Scripts: Discussion

- **Maintenance:** Every change in the build processes must be encoded in a linear script.
- **Readability:** Long build scripts tend to become difficult to read, especially when logic is introduced.
- **Standardization:** Build scripts are highly custom and require developer “ramp-up”.

⇒ This approach does not scale!

Declarative Builds



- Standardize common build operations
- Use a declarative build language
- Tools can abstract command syntax

```
package de.tudo.cs.ls14.aqua.boardgame;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import de.tudo.cs.ls14.aqua.boardgame.model.*;
import de.tudo.cs.ls14.aqua.boardgame.match.*;
import de.tudo.cs.ls14.aqua.boardgame.message.*;

/**
 * Message that business logic sends to UI
 */
public class Message {
    /**
     * enum MessageType (
     *     LOG,
     *     STATUS,
     *     GAME_OVER
     * )
     */
    private MessageType type;
    private String username;
    public MessageType getTypeId() {
        return type;
    }
    public void setTypeId(MessageType type) {
        this.type = type;
    }
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }
}
```

BoardGameApplication.java
Match.java
Message.java



```
javac -d build \
    src/**/*.java
jar cf dist/boardgame.jar \
    build/**/*.class
```



boardgame.jar

Demo: 02 – Ant

Declarative Builds: Discussion

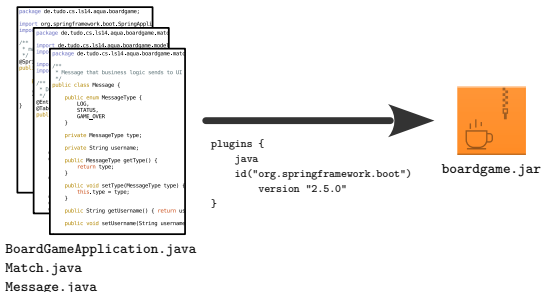
- **Maintenance:** The build configuration only needs to change if the build process changes or paths are modified.
- **Readability:** A domain-specific language allows developers to focus on the important parts; the files are comparatively succinct.
- **Standardization:** “Minutiae” such as source paths still change between projects.

⇒ Can we standardize even more?

Convention over Configuration



- Default to **conventional** paths
- Provide standard build definitions (e.g., `javac` → `jar`)
- Configuration is only necessary for special cases



Demos: 03 – Maven and 04 – Gradle

<https://maven.apache.org>, <https://gradle.com/brand/>, <https://github.com/KDE/breeze-icons>

Convention over Configuration: Discussion

- **Maintenance:** The build configuration almost never changes for a mature project.
- **Readability:** The files are even more succinct.
- **Standardization:** Normal projects can be built using a standard “template”.

⇒ Mission accomplished...?

Excursion: Reproducible Builds

“Reproducible builds are a set of software development practices that create an independently-verifiable path from source to binary code.”

- **Problem:** Even for open-source projects, trust in the binary files requires building from source or trusting the distributor to be honest.
- **Idea:** Permit external verification by making the build **reproducible**. If a third party can recreate the exact binary, the distributor is likely honest.
- **Problems:** Compiler optimizations, tool versions, timestamps, random values, ...

Adoption: TOR Project, many binaries in Debian, ...

Dependency Management

- **Project-specific:** Dependencies should be managed per project and be invisible to other projects on the same machine.
- **Versioned:** Specific versions of a dependency can be used, there is no automatic migration to the latest version (→ the developer becomes responsible for integrating security fixes in dependencies).
- **Recursive:** Dependencies of dependencies should be handled by the management system.
- **Automated:** Dependency installation should be as automated as possible. This may not be fully possible if, e.g., native libraries are involved.

Demos: 05 – Distribution and 06 – Dependencies

Centralized Repositories

- All dependencies come from a central server
- All (OSS) projects publish their software there
- Advantages: Central authority, clear chain of trust
- Problems: Disappearing dependencies, private artifacts
- ⇒ Local caches, additional repositories, ...

(* SOFTWARE *)

How one developer just broke Node, Babel and thousands of projects in 11 lines of JavaScript

Code pulled from NPM – which everyone was using

Chris Williams, Editor in Chief Wed 23 Mar 2016 // 01:24 UTC

SHARE

UPDATED Programmers were left staring at broken builds and failed installations on Tuesday after someone toppled the Jenga tower of JavaScript.

A couple of hours ago, Azer Koçulu unpublished more than 250 of his modules from **NPM**, which is a popular package manager used by JavaScript projects to install dependencies.

Koçulu yanked his source code because, we're told, one of the modules was called Kik and that apparently **attracted the attention of lawyers** representing the instant-messaging **app** of the same name.

Examples: Maven Central, NPM, PyPI, Docker Hub, ...

https://www.theregister.com/2016/03/23/npm_left_pad_chaos/

Build Tools as Dependencies

- Build tools are similar to libraries
- Manage using dependency management facilities
- ⇒ Developers do not need to install test frameworks etc.

```
package de.tudo.cs.ls14.aqua.boardgame;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

package de.tudo.cs.ls14.aqua.boardgame.match;

import de.tudo.cs.ls14.aqua.boardgame.match;
import de.tudo.cs.ls14.aqua.boardgame.match;

/**
 * Message that business logic sends to UI
 */
public class Message {
    /**
     * @enum
     * @attr
     * @attr
     * @attr
     */
    public enum MessageType {
        LOGIN,
        STATUS,
        GAME_OVER
    }

    private MessageType type;
    private String username;

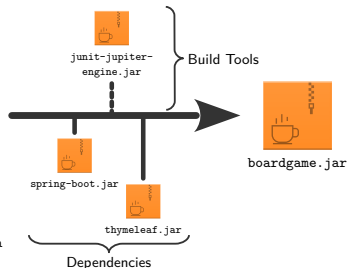
    public MessageType getTypew() {
        return type;
    }

    public void setType(MessageType type) {
        this.type = type;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

BoardGameApplication.java
Match.java
Message.java



Demo: 07 – Tests

Project Management: Examples

- **Code Quality:** enforce a code style during builds. IDEs can parse this to configure the formatter.
- **Legal Issues:** enforce a license header or check licenses of dependencies against a vetted list.
- **Documentation Generation:** Generate documentation such as JavaDoc or a manual. JavaDoc can even be distributed as a dependency!
- **Quality Metrics:** Check code coverage, known vulnerabilities in dependencies, etc.

Demo: 08 – Code Style

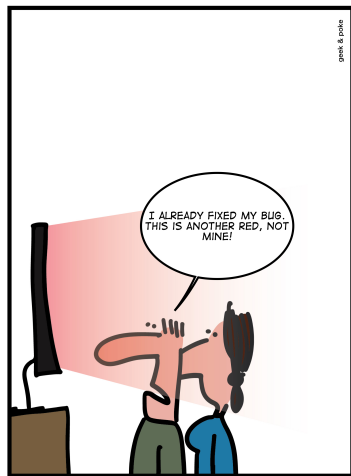
Agenda

1. Build Management

2. DevOps

3. Monitoring

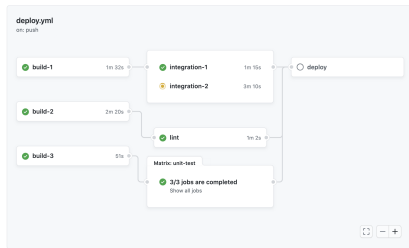
Breaking Builds



FIFTY SHADES OF RED

<https://geek-and-poke.com/geekandpoke/2017/2/15/fifty-shades-of-red>

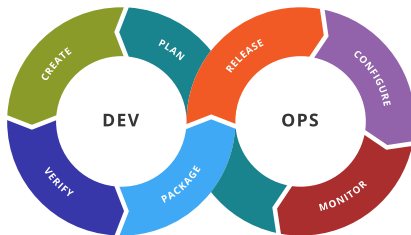
Continuous Integration: History



- **Idea:** execute long-running build and test tasks on a server over night (**nightly builds**)
- **Improvement:** perform builds whenever new code is committed (**continuously**)
- **Now:** combine with project management to check additional criteria

Continuous Deployment

- **Requirement:** sufficient quality control (CI + manual checks)
- Automatically perform releases / deployments
- Build server uploads to server, app store, update server, etc.



Demos: 11 – CD and Boardgame Server

Excursion: Canary Releases and Blue-Green-Deployment

- **Problem:** Automated deployments increase the risk of shipping faulty software. Issues may only arise on, e.g., a small number of devices.
- **Classic Approach:** Offer beta channels, early access programs, etc. This is generally not done by a **representative** subset of users.
- **Idea:** Release to a increasing set of users with a time delay. Example: 1% immediately, 33% after each day. Abort release if problems arise. Additional benefit: reduces load on servers!
- **Problems:** Keep parallel API versions, rollback plan for affected users, ...
- **Idea:** Keep the legacy version running (**blue deployment**) until the new version (**green deployment**) is stable.

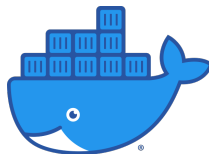
Containers

- **Dependency-Free:** A container includes all runtime dependencies of a piece of software. Only the container runtime is required to execute it.
- **Isolation:** The container is isolated from the host and other containers. This offers **some** security.
- **Stateless:** State is persisted over defined interfaces (e.g., a mounted database volume). The container itself is stateless.
- Other features: formalized configuration (e.g., exposed ports), orchestration with other containers, etc.

Examples: Docker, LXC, Snap, Apptainer, Flatpak, ...

Docker

- Container build language
- Linux-based (Windows + MacOS via VM)
- Multi-architecture support
- Central registry



Demos: 12 – Docker and 13 – CD plus Docker

Take-Home-Message

- **Build Management** automates the build operations, **Dependency** and **Project Management** the handling of libraries and build tools, respectively.
- **Continuous Integration** run project management tools on the VCS to ensure quality, and **Continuous Deployment** automates the release and delivery process.
- **Containers** pack an operating-system-like image of an application and its environment for easy, automated setup.
- Together, these enable **DevOps**, the fusion of development and operations.

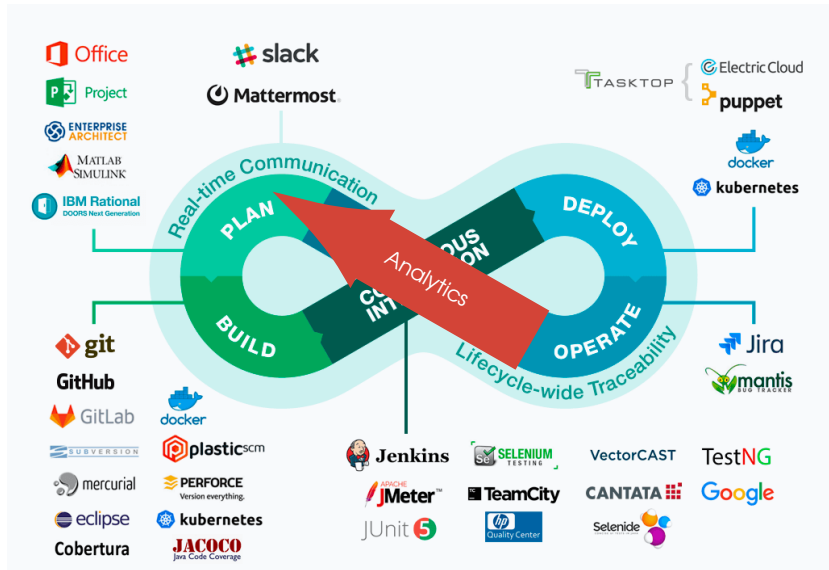
Agenda

1. Build Management

2. DevOps

3. Monitoring

Im Betrieb: Feedback der Kunden!



Analytics Frameworks



Beispiel aus Apple Ökosystem

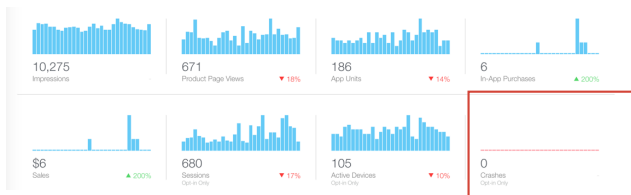
Daten aus:

- App-Store
- In-App Events

Gängige KPIs (wirtschaftlich)

- Retention rate
- Lifetime value (LTV)
- Uninstalls
- Cohort analysis
- ROI
- Cost per acquisition (CPA)
- User growth rate

Analytics Frameworks



Beispiel aus Apple Ökosystem

Daten aus:

- App-Store
- In-App Events

Technische Daten

- Crashes
- Click Paths
- Stack Traces
- Usage Times

Goal Question Metric Paradigm (GQM)

Goal (Purpose, Issue, Object/Process, Viewpoint):

(Konzeptuelle Ebene) Zweck, Aspekt, Gegenstand und Perspektive der Analyse

Question:

(Operative Ebene) Fragen, die das Untersuchungsobjekt in Bezug auf das Ziel charakterisieren

Metric:

(Quantitative Ebene) Daten zu jeder Frage (objektiv und subjektiv)

Beispiel:

Verbessern der Conversion Rate der RHC App aus Bannerwerbung bei X

Analyse:

- Wem wird die Werbung gezeigt?
- Wer klickt auf die Werbung?
- Wie bewegen sich Personen, die von X kommen auf unserer Seite?

Metrik / Maß:

- Anzeige Statistik von X
- Klick Statistik von X
- Klick Pfade für Besuche von X

<ftp://ftp.cs.umd.edu/pub/sel/papers/gqm.pdf>

Key Performance Indicators (KPIs)

- **Nutzer:** Täglich, monatlich, gesamt
- **Churn Rate/Retention Rate:** Rate der Nutzer die abwandern/bleiben
- **Umsatz pro Nutzer:** Wer zahlt wieviel
- **Transaktionen/Käufe:** Erfolg im App-Store
- **Conversion Rate:** Potentielle Kunden, die zu Kunden werden

Beispiel: <https://youtu.be/kkyFOU5Lyo0>

Weitere Metriken

- **Ladezeit/Reaktionszeit:** Zeit, bis die App/Seite geladen ist bzw.reagiert
- **Abstürze:** Anzahl, Frequenz, Gründe, Folgen
- **“Demographie”:** Alter, Herkunft der Nutzer, Art der Geräte
- **Nutzungsdauer:** Zeit pro Session in der App
- **Klick-Pfade:** Wege durch die App in Sessions

Beispiel: <https://youtu.be/kkyFOU5Lyo0>

A/B Testen

Einfacher praktischer Test einer Hypothese.

Observation

A

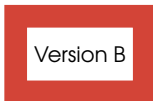


Conversion Rate: 30%

...

Click Rate: ...

B



Conversion Rate: 50%

...

Click Rate: ...

Angeblich von Google in über 7,000 Tests benutzt

A/B Testing: Statistische Grundlagen

A/B Testing = Statistischer Hypothesentest

- Nullhypothese (H_0): Es gibt keinen relevanten Unterschied zwischen A und B in Bezug auf Testgröße T
- Fehler erster Art: H_0 fälschlicherweise zurückweisen
- Fehler zweiter Art: H_0 fälschlicherweise beibehalten
- Signifikanzniveau (α): Wahrscheinlichkeit für Fehler erster Art
- p-Wert (p): Grenze für "statistisch signifikanten Unterschied" ($p = \text{Prob}(T \geq t)$)

⇒ Wir können H_0 ablehnen, wenn $p \leq \alpha$ ist.